Reporte

Nombre: Roberto Carlos Cantu González	**Matrícula**: 2114360
Nombre: Ángel Daniel Jiménez Fuantos	**Matrícula**: 2117083
Nombre: Mario Ivan González Martínez	**Matrícula**: 2117170
Nombre del curso: Componentes de Lógica de Negocios	**Nombre del profesor**: Miguel Ángel Gómez Marroquín
Modulo Netacad: 3	**Actividad**: Actividad 3. ADR
Fecha: 15/10/2026	

Objetivo:

En esta actividad convertirán el análisis de negocio elaborado en la Unidad 2 en una propuesta de arquitectura de software. Trabajarán con la misma empresa y el mismo caso. La inteligencia artificial podrá apoyar el diseño y la revisión, pero cada decisión deberá justificarse con evidencia del proyecto.

El objetivo es organizar las reglas y responsabilidades del sistema antes de programarlo por completo. Al terminar, el equipo deberá demostrar cómo pasó de los requerimientos de la Unidad 2 a una arquitectura que separa el dominio, la aplicación y la infraestructura.

La actividad no consiste en pedir a una herramienta de IA que produzca una solución terminada. El trabajo consiste en plantear decisiones, solicitar alternativas, revisar las respuestas y documentar por qué una propuesta sí corresponde o no al negocio analizado.

Architecture Decision Records

ADR-01: Selección del Patrón Domain Model para la Lógica de Negocio

- **Título:** ADR-01 Selección del Patrón Domain Model.
- **Estatus:** Aceptado.
- **Contexto:** TecnoStock opera 5 sucursales físicas y comercializa electrónicos con trazabilidad por número de serie. La operación requiere garantizar que la mercancía apartada no se venda por error en mostrador, que los anticipos respeten el límite mínimo del 20% y que las devoluciones estén estrictamente validadas. Diseñar la lógica mediante scripts o tablas relacionales generaba riesgo de duplicación de reglas y falta de protección sobre el inventario reservado.
- **Alternativas Consideradas:**
    1. *Transaction Script:* Descartado por el alto riesgo de duplicación de código en terminales de caja y falta de encapsulamiento para proteger el estado del inventario.
    2. *Table Module:* Descartado por su rigidez para representar operaciones orientadas a objetos y coordinar reglas entre múltiples entidades.
    3. *Domain Model:* Seleccionado por encapsular el estado y el comportamiento en objetos de dominio ricos (Aggregates).
- **Decisión:** Adoptar el patrón **Domain Model** como patrón central de arquitectura para la lógica de negocio de TecnoStock.
- **Consecuencias:**
    - *Beneficios:* Encapsulamiento total de invariantes de negocio (ej. la reserva de ItemSerie y la regla del 20% de Apartado se ejecutan de forma segura dentro del dominio), facilidad para realizar pruebas unitarias sobre reglas sin depender de la base de datos.
    - *Costos/Riesgos:* Curva de aprendizaje inicial y mayor cantidad de clases e interfaces en la solución.
- **Trazabilidad:** N-01, N-02, N-03, US-02, US-03, US-04.

ADR-02: Separación de Responsabilidades mediante CQRS Básico

- **Título:** ADR-02 Separación de Commands y Queries (CQRS Básico).
- **Estatus:** Aceptado.
- **Contexto:** La restricción **CON-01** exige que las búsquedas de disponibilidad entre las 5 sucursales respondan en menos de 3 segundos en punto de venta. Cargar el modelo completo de dominio con todas sus entidades e historial para realizar consultas de solo lectura ralentizaba las cajas de cobro.
- **Alternativas Consideradas:**
    1. *Modelo Unificado (Domain Model para Lectura y Escritura):* Descartado por sobrecargar la memoria en cajas y violar el principio SRP.
    2. *CQRS Básico (Separación de Handlers y DTOs de Lectura):* Seleccionado para optimizar las consultas sin alterar la base de datos física.
- **Decisión:** Implementar **CQRS básico** en la capa de aplicación. Las modificaciones de estado se ejecutan mediante *Commands* que pasan por el modelo de dominio, mientras que las consultas utilizan *Queries* que leen directamente de vistas optimizadas (*Read Models*).
- **Consecuencias:**
    - *Beneficios:* Tiempos de respuesta de lectura < 3 segundos en la red de tiendas, DTOs de lectura diseñados a la medida de la pantalla de la caja.
    - *Costos/Riesgos:* Mantenimiento de dos estructuras de clases separadas para lectura y escritura.
- **Trazabilidad:** CON-01, N-01, US-01.

ADR-03: Abstracción de Persistencia con Repository y Unit of Work

- **Título:** ADR-03 Estrategia de Persistencia y Transacciones.
- **Estatus:** Aceptado.
- **Contexto:** El núcleo del dominio de TecnoStock no debe acoplarse a librerías ORM específicas ni a motores SQL. Además, operaciones como registrar un apartado requieren actualizar la entidad Apartado y bloquear el agregado ItemSerie en una misma transacción atómica.
- **Alternativas Consideradas:**
    1. *Invocaciones directas a SQL desde los servicios:* Descartado por violar DIP y acoplar la aplicación a la base de datos.
    2. *Repository Pattern + Unit of Work:* Seleccionado para desacoplar el acceso a datos y garantizar transacciones atómicas.
- **Decisión:** Definir interfaces puras de IRepository en el Dominio e implementarlas en la capa de Infraestructura, utilizando IUnitOfWork para gestionar el inicio, Commit() y Rollback() de transacciones.
- **Consecuencias:**
    - *Beneficios:* Independencia tecnológica total, aislamiento de transacciones ACID y capacidad de simular repositorios en memoria para pruebas.
    - *Costos/Riesgos:* Introducción de interfaces y mappers adicionales entre el modelo relacional y las entidades de dominio.
- **Trazabilidad:** CON-03, N-03, US-02, US-03.